

EC-231 Operating Systems

BS(CE)-2021

LAB REPORT # 08



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-021	Yawar Ali Malik
21-CE-020	Muhammad Ibthesam

Department of Computer Engineering

BS Computer Engineering Program

HITEC University Taxila

	Presentation (Format (0.5) + Title (0.5) + Conclusion(1) + Procedure(2) + Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.
Engr. Fasih Ahmad

Experiment # 08

Process related System Calls in Linux

Objective

In this lab we will learn about system call techniques i.e.

- ☐ fork()
- ☐ exit()
- ☐ wait()
- ☐ exec()

Software Used

Virtual Box

Ubuntu

Procedure

1. Using fork() and exit():

Open gedit editor.

Write a suitable command to create a child process using fork() and execute a command.

Implement the child process to execute a command.

Use exit() to gracefully terminate the child process.

Save and run the script to observe the execution.

2. Using wait() and exit():

Open gedit editor.

Write a suitable command to create a child process using fork() and execute a command.

Implement the parent process to wait for the child process to complete using wait().

Use exit() to gracefully terminate both parent and child processes.

Save and run the script to observe the execution.

3. Task 3

Open gedit editor.

Write C code based on Algorithm 1:

Declare variables.

Implement parent and child processes using fork().

Print process IDs and statuses.

Exit both processes.

Write C code based on Algorithm 2:

Declare variables.

Print a "Hello World".

Implement parent and child processes using fork().

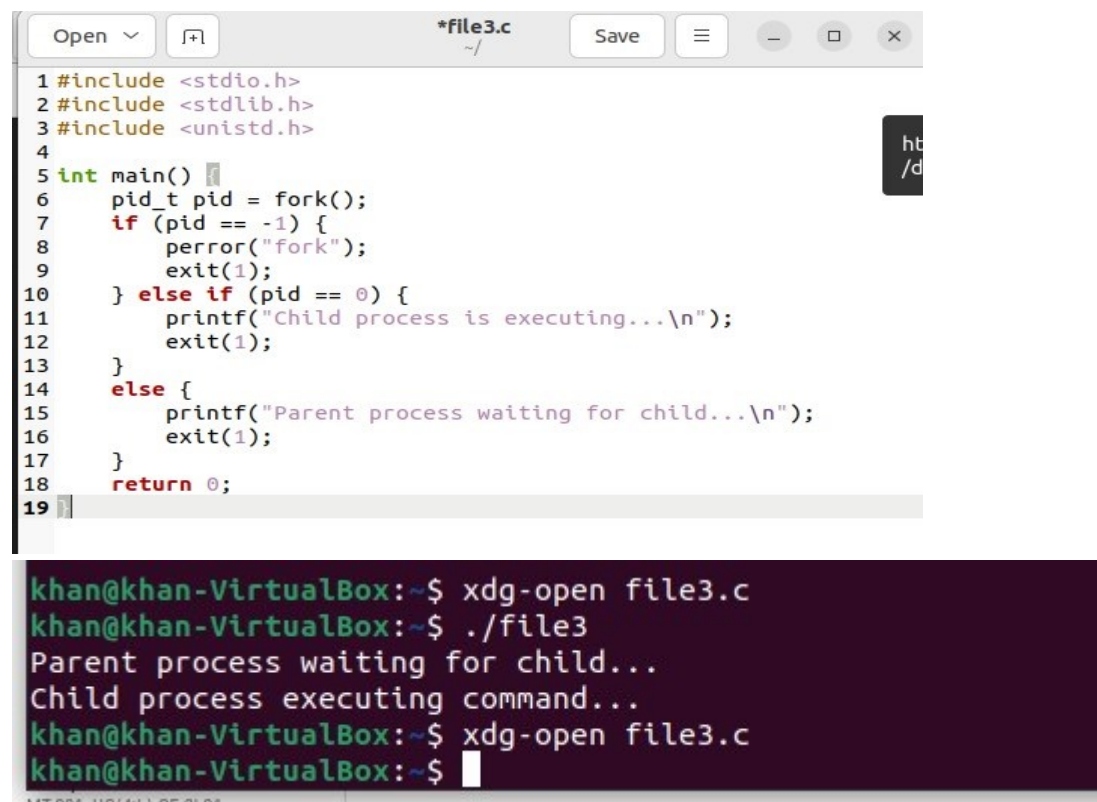
Print process IDs and statuses.

Gracefully exit both processes.

Save and compile the C programs.

Tasks

Task 1



```
Open  [+1]  *file3.c  Save  [Menu]  [Zoom In]  [Zoom Out]  [Close]

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     pid_t pid = fork();
7     if (pid == -1) {
8         perror("fork");
9         exit(1);
10    } else if (pid == 0) {
11        printf("Child process is executing...\n");
12        exit(1);
13    }
14    else {
15        printf("Parent process waiting for child...\n");
16        exit(1);
17    }
18    return 0;
19 }
```

```
khan@khan-VirtualBox:~$ xdg-open file3.c
khan@khan-VirtualBox:~$ ./file3
Parent process waiting for child...
Child process executing command...
khan@khan-VirtualBox:~$ xdg-open file3.c
khan@khan-VirtualBox:~$
```

Task 2

```
Open  [icon]  *file3.c  Save  [icon]  [icon]  [icon]
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t pid = fork();
8     if (pid == -1) {
9         perror("fork");
10        exit(1);
11    } else if (pid == 0) {
12        printf("Child process executing command...\n");
13        exit(1);
14    }
15    else {
16        printf("Parent process waiting for child...\n");
17        wait(NULL);
18        printf("Parent process exiting...\n");
19        exit(1);
20    }
21    return 0;
22 }
```

```
khan@khan-VirtualBox:~$ xdg-open file3.c
khan@khan-VirtualBox:~$ ./file3
Parent process waiting for child...
khan@khan-VirtualBox:~$ Child process executing command...
xdg-open file3.c
khan@khan-VirtualBox:~$
```

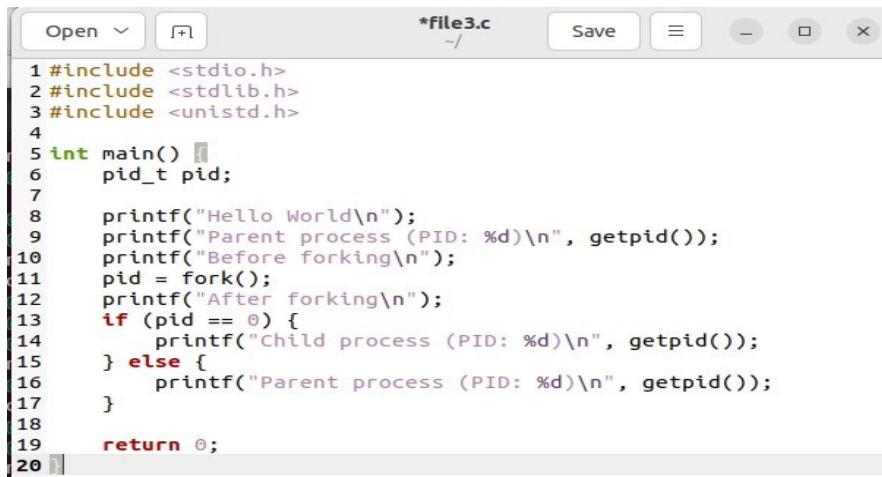
Task 3

Algorithm 1:

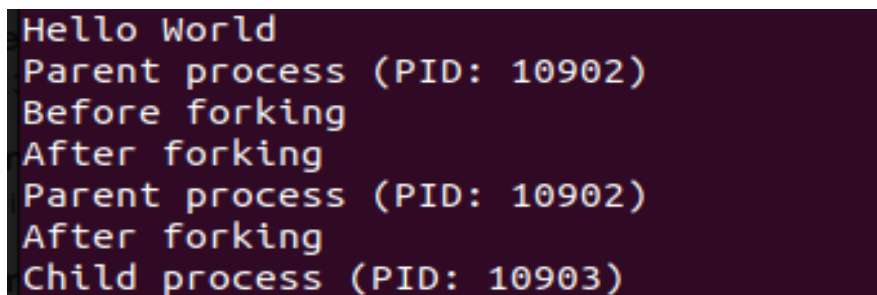
```
Open  [icon]  file3.c  Save  [icon]  [icon]  [icon]
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     pid_t forkresult;
7
8     printf("Parent process (PID: %d)\n", getpid());
9     printf("During forking (PID: %d)\n", getpid());
10
11    forkresult = fork();
12
13    if (forkresult != 0) {
14        printf("Child process (PID: %d)\n", forkresult);
15    } else {
16        printf("Child process (PID: %d)\n", getpid());
17    }
18
19    printf("Parent process (PID: %d)\n", getpid());
20
21    return 0;
22 }
```

```
Parent process (PID: 10491)
During forking (PID: 10491)
Child process (PID: 10492)
Parent process (PID: 10491)
Child process (PID: 10492)
Parent process (PID: 10492)
```

Algorithm 2:



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     pid_t pid;
7
8     printf("Hello World\n");
9     printf("Parent process (PID: %d)\n", getpid());
10    printf("Before forking\n");
11    pid = fork();
12    printf("After forking\n");
13    if (pid == 0) {
14        printf("Child process (PID: %d)\n", getpid());
15    } else {
16        printf("Parent process (PID: %d)\n", getpid());
17    }
18
19    return 0;
20 }
```



```
Hello World
Parent process (PID: 10902)
Before forking
After forking
Parent process (PID: 10902)
After forking
Child process (PID: 10903)
```

Conclusion

In this lab, we delved into the features of `wait()`, `fork()`, and `exit()` system calls, enriching our comprehension of process control. By mastering the art of coordinating processes, creating child processes, and overseeing their completion, we've refined indispensable competencies essential for crafting resilient Unix-oriented applications.